The GRASS Temporal Framework: Object oriented code design with examples

Sören Gebbert

GRASS 8.6, 6 November 2026

Abstract

This document is a revised version of the supplementary material of Sören Gebbert and Edzer Pebesma: The GRASS GIS temporal framework. International Journal of Geographical Information Science 31(7), 1273–1292, 2017, doi:10.1080/13658816.2017.1306862 [3]. It describes the code as it is in GRASS 8.6. The version of 2017, by Sören Gebbert and Edzer Pebesma, lies unchanged next to this document as `Temporal-Framework-API-Description.pdf`; it shows the code of 2017.

Contents

1 Introduction **1**

2 Object oriented code design **2**
2.1 Rows of the temporal database . . . 2
2.2 Abstract datasets and the topology . . . 2
2.3 Map and space time dataset classes . . . 4
2.4 Registered maps as lists of objects . . . 7

3 The database layer **9**

4 The algebra core and the classic algebra **9**

5 Code examples **10**
5.1 Registration of raster maps . . . 10
5.2 Temporal aggregation . . . 12
5.3 Aggregation with the algebra core . . . 13
5.4 Sampling a raster time series at vector points . . . 14
5.5 Spatio-temporal intersection of trajectories . . . 16
5.6 Benchmark . . . 19

1 Introduction

The temporal framework of GRASS manages time series of raster, 3D raster and vector maps [2, 3]. A map gets a time stamp, and maps with time stamps are registered in space time datasets: space time raster datasets (STRDS), space time 3D raster datasets (STR3DS) and space time vector datasets (STVDS). The time stamps and the spatial extents of maps and datasets are kept in a temporal database, by default an SQLite database in the mapset (`tgis/sqlite.db`). The temporal tools (`t.*`) are written in Python on top of the Python package `grass.temporal`, except `t.connect`, which is written in C.

This document describes the classes of `grass.temporal` (section 2), the database layer below them (section 3) and the algebra core, which computes the temporal algebra, the aggregations, the map calculations and the sampling of the tools (section 4). Section 5 shows Python examples: the registration of maps, temporal aggregation, the same aggregation with the algebra core, sampling at vector points, the spatio-temporal intersection of trajectories and a benchmark.

Every example is a Python file of the GRASS repository, in the folder `examples` of the source of this document, `python/grass/docs/src/temporal_framework_design`; the caption of each listing gives the path of its file. Each file is also attached to this PDF under its name, so that a PDF viewer can save it from its list of attachments. The examples run in the North Carolina sample dataset `nc_spm_08_grass7` [4], available from https://grass.osgeo.org/download/data/, in a new mapset `temporal_design`. They make their

own data from maps of the dataset and never write into its mapset `PERMANENT`. To run them, save the files and start them in the order of section 5, the first one with `-c`, which creates the mapset:

```
grass -c ~/grassdata/nc_spm_08_grass7/temporal_design --exec python3 register.py
grass ~/grassdata/nc_spm_08_grass7/temporal_design --exec python3 aggregate.py
```

The tests of GRASS run each example in such a mapset and compare its output with the output that this document shows; they check also that each class and member that a figure draws exists in the code.

2 Object oriented code design

Maps and space time datasets are Python objects whose content is stored in the temporal database. Each object keeps its content in rows, one row per table: the base information, the absolute or relative time, the spatial extent, the metadata and a register. The figures of this section show the classes in UML, simplified: they show the members that a user of the class needs most, not all of them.

2.1 Rows of the temporal database

Figure 1 shows the classes of the rows. `DictSQLSerializer` holds the content of a row in the dictionary `D` and turns it into the SELECT, INSERT and UPDATE statements of a table and back. `SQLDatabaseInterface` names the table and the identifier of the row and runs the statements: `select()`, `insert()`, `update()`, `update_all()`, `delete()` and `is_in_db()`. The identifier of a map is `name@mapset`, of a vector map with a layer `name:layer@mapset`, because a time stamp can be given to each layer of a vector map.

`Row` lists its columns once, as `Column` objects in the class attribute `columns`; the getter, the setter and the property of each column are generated when the class is created. The rows of a map or dataset are `BaseRow` (identifier, name, mapset, creator, creation time, temporal type), `RegisterRow` (the datasets a map is registered in), `AbsoluteTimeRow` or `RelativeTimeRow` (the time stamp), `SpatialExtentRow` (the spatial extent) and a metadata row. One class serves all six types of maps and datasets: a `Kind` (`kinds.py`) tells the row its table and which columns it has. The modification time and the semantic type of `BaseRow`, and the granularity and the temporal type of the maps (`map_time`) of the time rows, are columns of space time datasets only.

`TemporalExtent` computes the temporal relations between two time stamps (the relations of Allen [1] and their names in GRASS, such as `starts()`, `during()`, `follows()` or `overlaps()`, together with `temporal_relation()`, which names the relation), and the intersection and the unions of two time stamps. `SpatialExtent` does the same for spatial extents in two and three dimensions; the methods with `_2d` use north, south, east and west only.

The metadata differ by type (figure 2). The six metadata classes are generated from tables of columns in `metadata.py`; `MetadataRow(kind)` returns an object of the class of the kind, so that a raster map has no number of points and a vector map no semantic label. The metadata of a space time dataset hold the extremes of the metadata of its maps: the smallest and largest minimum (`min_min`, `min_max`), the smallest and largest maximum (`max_min`, `max_max`) and the smallest and largest resolution in each direction (`nsres_min` to `tbres_max`), because maps of different resolution can be registered in one dataset. The mixins `_SpaceTimeMetadata` and `_SemanticLabelsMetadata` add what the datasets share.

2.2 Abstract datasets and the topology

Maps and space time datasets share an abstract base class, `AbstractDataset` (figure 3). It reads the time stamp and the spatial extent of an object, computes temporal relations and intersections, and selects, inserts, updates and deletes the object in the temporal database. The spatial methods are abstract: two mixins implement them for two- and three-dimensional extents (figures 4 and 5).

`AbstractDataset` inherits from `TopologyLinks`, which keeps the links of an object to the objects it is temporally and spatially related to: one dictionary per kind of relation, with a list of objects for each relation, and the temporal successor and predecessor (`next()`, `prev()`). The `SpatioTemporalTopologyBuilder` sets these links. It builds the topology of one list of objects, or between two lists, with the R*-tree of the GRASS vector library, and it links the objects of each list in the order of their start times by `next()` and `prev()`. An object that starts or finishes another one is also entered as during it, and an object that is started or finished by another one also contains it, so `get_contains()` of a month lists every day of the month, the first and the last day included.

`AbstractMapDataset` sets the time stamp of a map, buffers its time stamp and its spatial extent, and keeps the register of the datasets a map is registered in (`get_registered_stds()`, `add_stds_to_register()`,

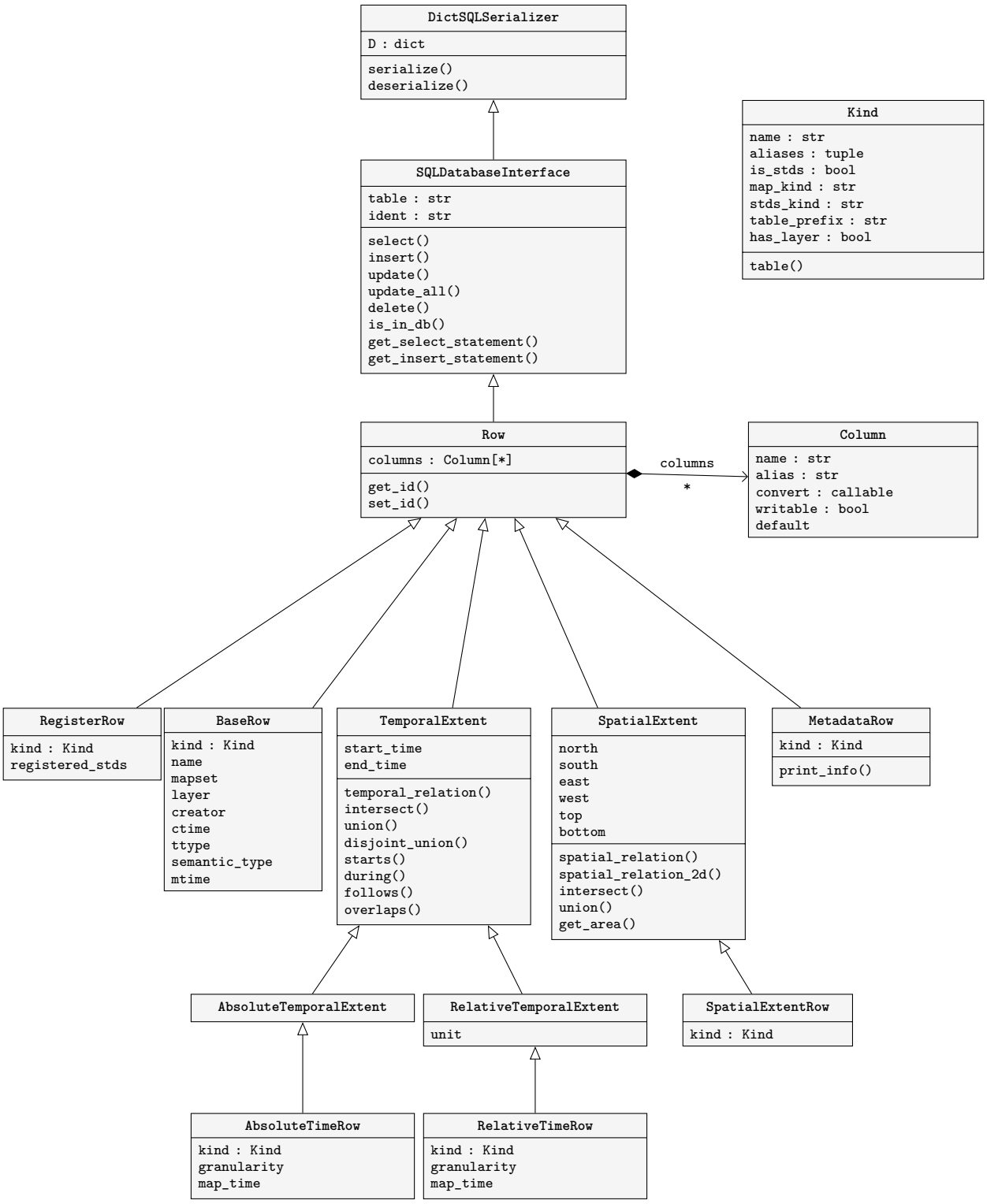


Figure 1: The row classes. Every row has the identifier of its map or dataset and the name of its table; a `Kind` chooses table and columns, so one class serves raster, 3D raster and vector maps and their space time datasets.

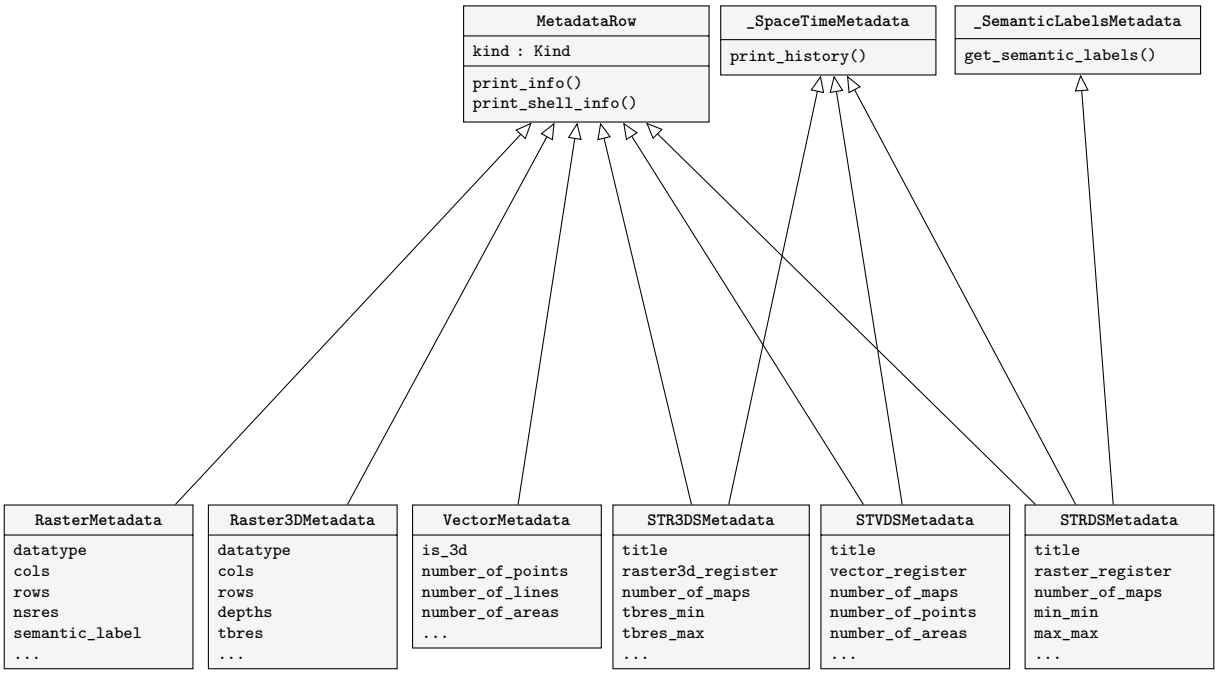


Figure 2: The metadata classes, generated from tables of columns. A list of columns that ends in "..." is shortened.

`remove_stds_from_register()`); `delete()` removes a map from every dataset it is registered in. `MapDataset` implements the methods that read a map: `load()` asks the C library for the metadata of the map, in a separate process (`c_libraries_interface.py`), so that an error of the C library does not end the caller.

`AbstractSpaceTimeDataset` registers and unregisters maps, updates the dataset from its maps (`update_from_registered_maps()`), computes the granularity and the temporal topology of its maps, and shifts and snaps their time stamps. Shifting moves all time stamps by a granule, such as one year, two weeks or five minutes. Snapping sets the end of each map to the start of the next one, except for maps that start together; a last map without an end time gets one granule of the granularity of the dataset as its length, a last map with an end time keeps it. The register of the maps of a dataset is a table of its own, which `StdsMapRegister` creates, fills and reads.

2.3 Map and space time dataset classes

The classes of the three map types and of the three space time dataset types (figures 4 and 5) combine `MapDataset` or `StdsDataset` with a mixin for the spatial extent, `_PlanarExtentMethods` for two dimensions or `_VolumeExtentMethods` for three. A class sets its `Kind`; a map class also copies what the C library reports about a map of its type into the metadata. The rows are made by `reset()`, which the constructor calls with the identifier. Of the classes of these figures, the six classes of the types are exported from `grass.temporal`; `MapDataset`, `StdsDataset` and the mixins are not.

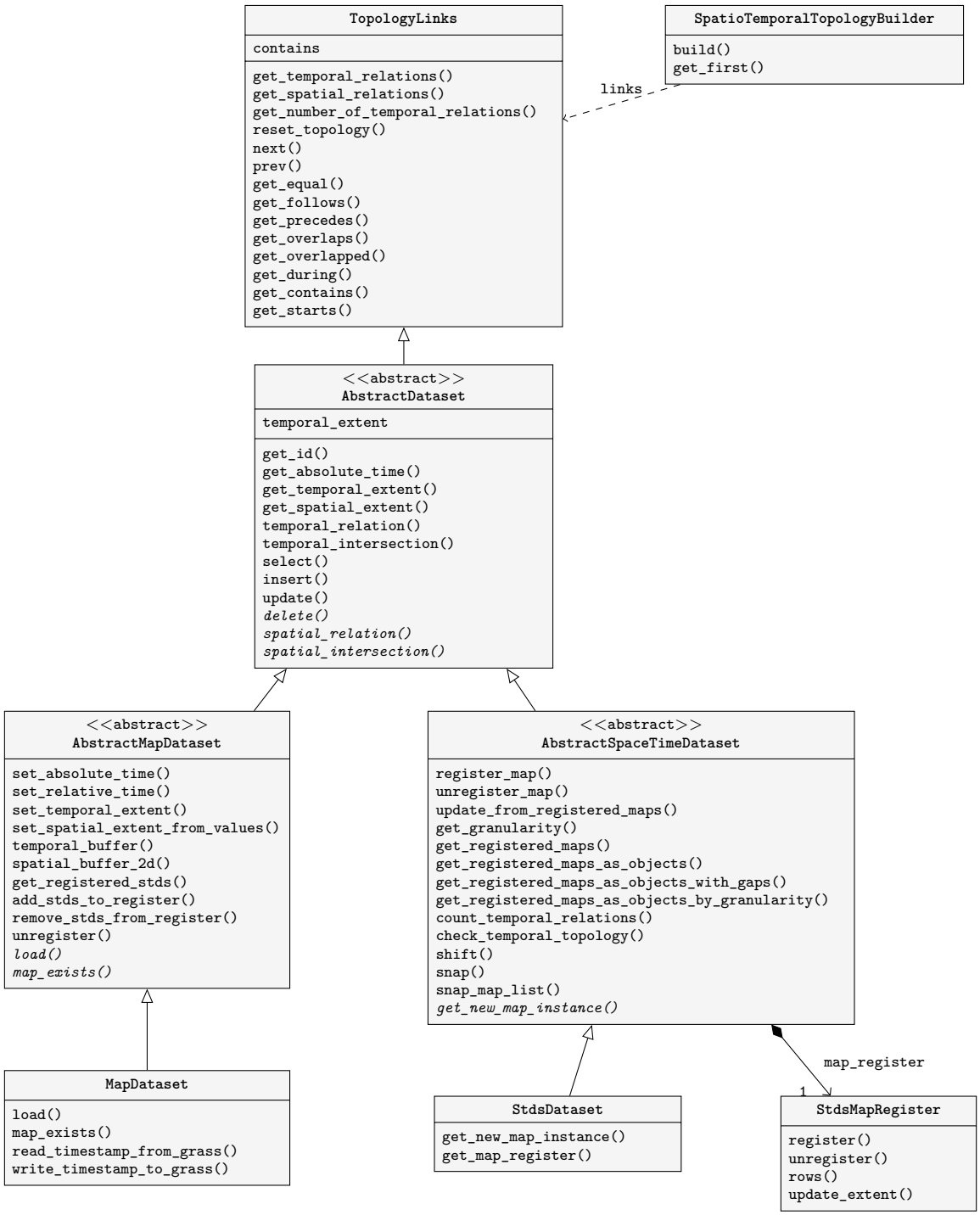


Figure 3: The abstract dataset classes. Abstract methods are in italics. The topology builder links the objects of one or two lists; the register of the maps of a space time dataset is a table of its own.

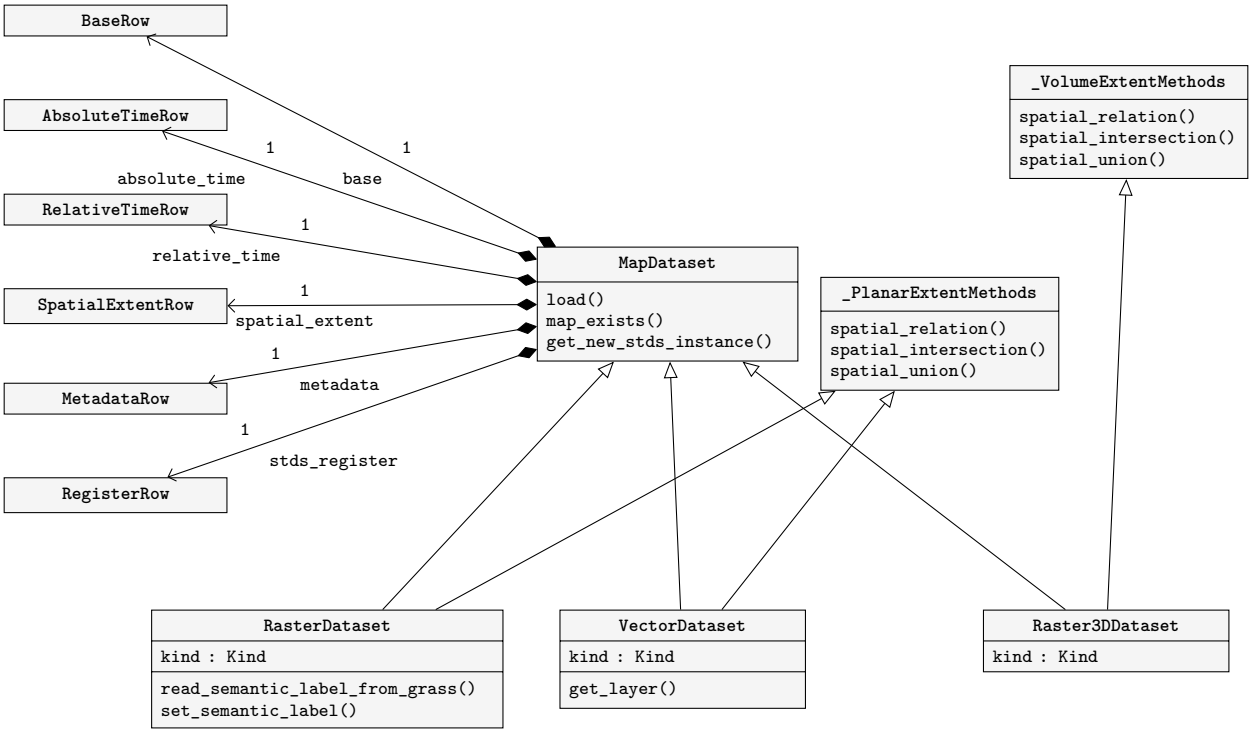


Figure 4: The map classes and the rows of a map.

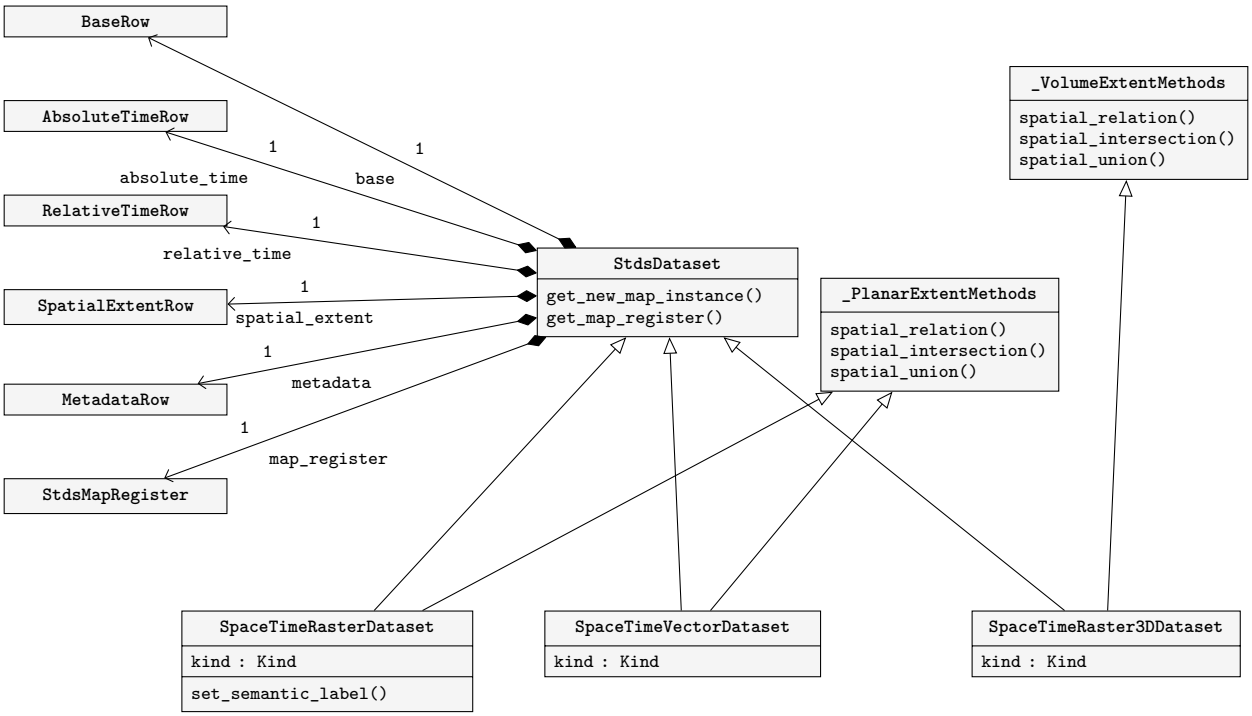


Figure 5: The space time dataset classes and the rows of a dataset. A dataset has the register of its maps instead of a register of datasets.

2.4 Registered maps as lists of objects

Three methods of a space time dataset return its maps as objects. The objects carry the identifier, the time stamp and the spatial extent of a map, which is what the topology needs; `select()` reads the rest. Figures 6 to 8 show the result for a dataset of three maps of absolute time with gaps between them: A from 20 to 22 January 2010, B from 24 to 27 January and C from 28 to 29 January. The granularity of the dataset is one day. The arrows show the order of the list. They are not links between the objects: these methods set no `next()` or `prev()`. Listing 1 creates the dataset and prints the three lists.

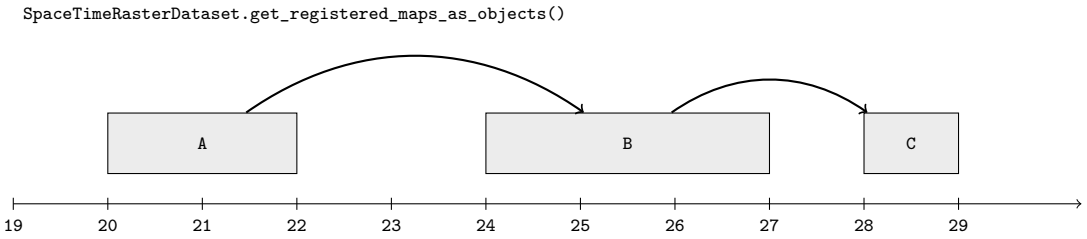


Figure 6: The maps of the dataset ordered by start time. The arrows show the order of the list; they are not links between the objects, the method sets no `next()` or `prev()`.

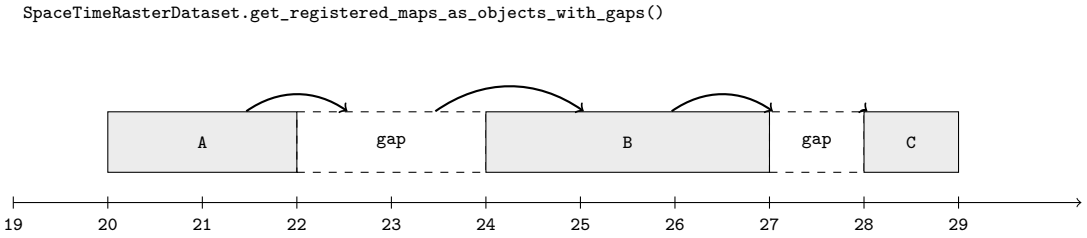


Figure 7: The maps with a gap object for each gap. A gap is an object of the class of the maps, here `RasterDataset`, with the identifier `None` and the time of the gap. The arrows show the order of the list; they are not links between the objects, the method sets no `next()` or `prev()`.

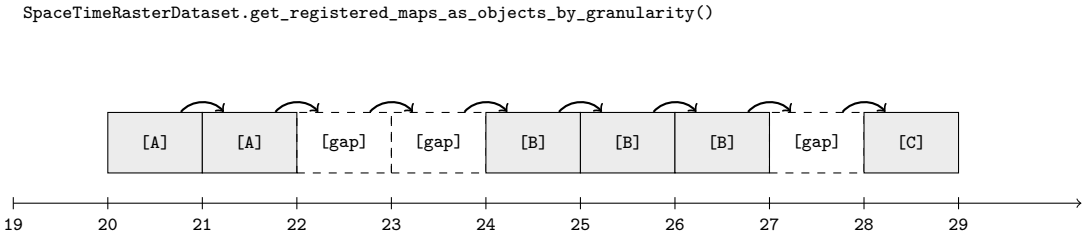


Figure 8: One list per granule of the granularity. Each list holds a new object for each map in the granule, with the identifier and the spatial extent of the map and the time of the granule, or a gap object. The arrows show the order of the lists; they are not links between the objects, the method sets no `next()` or `prev()`. The method needs a valid temporal topology: maps must not overlap or contain each other.

Listing 1: The dataset of figures 6 to 8
`python/grass/docs/src/temporal_framework_design/examples/map_lists.py`
attached to this PDF as `map_lists.py`

```
"""Register three maps with gaps and list them in the three ways of the document"""

from datetime import datetime

import grass.temporal as tgis
from grass.tools import Tools

tools = Tools()
tools.g_region(raster="elev_state_500m", res=1000, flags="a")
```

```
tools.r_mapcalc(expression="A = 1\nB = 2\nC = 3")

tgis.init()
dataset = tgis.open_new_stds(
    name="abc",
    type="strds",
    temporaltype="absolute",
    title="Three maps",
    descr="Three maps with gaps between them",
    semantic="mean",
    overwrite=True,
)
mapset = tgis.get_current_mapset()
times = {
    "A": (datetime(2010, 1, 20), datetime(2010, 1, 22)),
    "B": (datetime(2010, 1, 24), datetime(2010, 1, 27)),
    "C": (datetime(2010, 1, 28), datetime(2010, 1, 29)),
}
for name, (start, end) in times.items():
    layer = tgis.RasterDataset("{}@{}".format(name, mapset))
    layer.load()
    layer.set_absolute_time(start, end)
    layer.insert()
    dataset.register_map(layer)
dataset.update_from_registered_maps()
print("Granularity:", dataset.get_granularity())


def describe(layer):
    """The name, or "gap" for a gap, and the time of a map object"""
    start, end = layer.get_absolute_time()
    name = layer.get_name() if layer.get_id() else "gap"
    return "{} {:%m-%d}..{:%m-%d}".format(name, start, end)


print("get_registered_maps_as_objects():")
print(", ".join(describe(layer) for layer in dataset.get_registered_maps_as_objects()))
print("get_registered_maps_as_objects_with_gaps():")
print(
    ", ".join(
        describe(layer) for layer in dataset.get_registered_maps_as_objects_with_gaps()
    )
)
print("get_registered_maps_as_objects_by_granularity():")
for granule in dataset.get_registered_maps_as_objects_by_granularity():
    print("[{}]".format(", ".join(describe(layer) for layer in granule)))
```

Listing 2: Output of `map_lists.py`

```
Granularity: 1 day
get_registered_maps_as_objects():
A 01-20..01-22, B 01-24..01-27, C 01-28..01-29
get_registered_maps_as_objects_with_gaps():
A 01-20..01-22, gap 01-22..01-24, B 01-24..01-27, gap 01-27..01-28, C 01-28..01-29
get_registered_maps_as_objects_by_granularity():
[A 01-20..01-21]
[A 01-21..01-22]
[gap 01-22..01-23]
[gap 01-23..01-24]
[B 01-24..01-25]
[B 01-25..01-26]
[B 01-26..01-27]
[gap 01-27..01-28]
[C 01-28..01-29]
```

3 The database layer

The database layer consists of five modules:

state `init()` finds out everything about the session once: the database backend, the database, the mapset, the messenger and the C library interface. All of it lives in one object, `STATE`; the functions `get_*()` read it, and only `init()` and `set_raise_on_error()` write it. Unless it is called with `skip_db_init`, `init()` creates the temporal database if it does not exist and stops at a database of a version that the framework does not read; `t.upgrade` upgrades an older one. Every script calls `init()` before it uses the framework.

db `SQLDatabaseInterfaceConnection` holds the connections to the temporal databases of the mapsets in the search path, one `DBConnection` per database; a mapset outside the search path joins when a dataset or map named with it is first read. The backend is SQLite (`sqlite3`) or PostgreSQL (`psycopg2`). The functions `sql_identifier()` and `sql_statement()` check every table, view and column name that goes into the text of a statement; values are passed as parameters or quoted by `sql_literal()`.

schema creates the tables and views of the temporal database from the SQL templates, reads its version from its metadata table and upgrades it (`t.upgrade`).

kinds names the six types in one place: the names a user may type, the tables and views of each type, the prefix of the functions of the C library interface, the type of `g.remove` and the module that computes a map of the type from an expression.

rows the row classes of figure 1.

A method that reads or writes the database takes an optional `dbif`. Without it, the method opens a connection, runs its statements and closes the connection again; a script that runs many statements opens one connection and passes it on (listing 3). `register_maps_in_space_time_dataset()` always opens a connection of its own; its argument `dbif` is deprecated.

4 The algebra core and the classic algebra

The algebra core is the package `grass.temporal.algebra`. It computes the expressions of `t.rast.algebra`, `t.rast3d.algebra`, `t.vect.algebra` and `t.select`, the aggregations of `t.rast.aggregate` and `t.rast.aggregate.ds`, the map calculations of `t.rast.mapcalc` and `t.rast3d.mapcalc` and the sampling of `t.sample` and `t.vect.observe.strds` (figure 9). A series is an immutable, time ordered sequence of slots. A slot has a time stamp (`Extent`), optionally a spatial extent (`Space`), and a content tree that says what the map of the slot contains: an existing map (`MapRef`), a constant (`Const`), a function of the raster algebra (`Call`) or a reduction of several maps (`SeriesReduce`, `SeriesAggregate`). Building a series computes no map.

The operations are functions from series to series: `select()` keeps the slots that are related to slots of another series, `pairs()` combines related slots, `gather()` reduces the slots related to each slot of a target, `apply()` and `where()` combine series slot by slot. A `Relation`, such as `contains` or `during|starts`, says which slots are related; the core asks the topology builder of `grass.temporal`. `source()` reads a space time dataset as a series and `granules()` makes empty slots on a regular grid of time. Only `materialize()` computes maps: it turns the content trees into calls of `r.mapcalc`, `r3.mapcalc`, `r.series`, `g.copy`, `v.overlay`, `v.patch` or `v.buffer` and registers the maps in a new space time dataset. `plan()` returns the same calls as a `ProcessChain` without running them.

The parser classes of the tools, such as `TemporalRasterAlgebraParser`, read an expression with the grammar of the core and a translator, which turns the syntax tree into calls of the operations.

The classic algebra is the language of the temporal algebra as the manuals and the publications define it. The core computes its expressions; the manuals of the algebra tools list where the results differ from the releases before GRASS 8.6. The implementation that is older than the core stays in `grass.temporal.classic` as the reference implementation, and the tests compare the core with it.

Listing 7 computes the monthly means of listing 5 with the core.

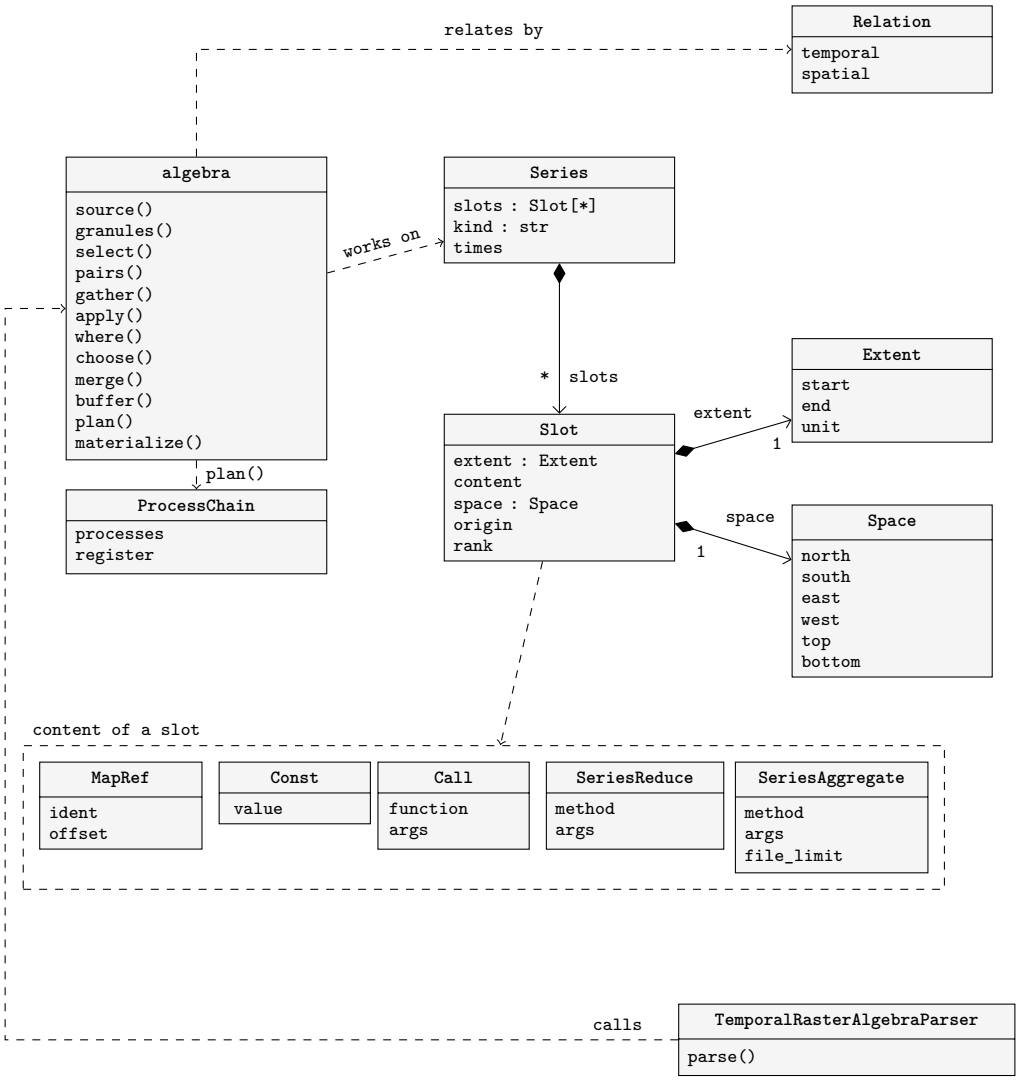


Figure 9: The algebra core. The box `algebra` is the package `grass.temporal.algebra` with some of its functions; the other boxes are classes. Series, slots and content trees are immutable values.

5 Code examples

The examples run in the mapset `temporal_design` of the North Carolina sample dataset, in the order of this section; an example names the examples it needs in its docstring. Run times depend on the computer; the output does not, except for the times that the benchmark writes to standard error. The tests compare the output of each example with the output shown here.

5.1 Registration of raster maps

Listing 3 computes 366 maps of the daily mean air temperature of 2020 with one call of `r.mapcalc`. The values are generated, not measured, and stored as whole hundredths of a degree Celsius, as the gridded E-OBS data that the version of 2017 used. Each map is loaded as a `RasterDataset`, gets its day as time stamp, is inserted into the temporal database and registered in a new STRDS. One database connection serves all statements.

Listing 3: Registration of daily temperature maps
`python/grass/docs/src/temporal_framework_design/examples/register.py`
attached to this PDF as `register.py`

```
"""Create daily temperature maps of 2020 and register them in a new STRDS

The maps are generated from the elevation model of the North Carolina sample
dataset and stored in hundredths of a degree Celsius, as integers.
"""
```

```
from datetime import datetime, timedelta

import grass.temporal as tgis
from grass.tools import Tools

tools = Tools()
tools.g_region(raster="elev_state_500m", res=1000, flags="a")

# Daily mean air temperature: 16.5 degree Celsius at sea level, an annual
# cycle of 10.5 degree with its maximum in mid July, a weather term of 3
# degree, -6.5 degree per 1000 m and +1 degree per 100 km to the south.
first_day = datetime(2020, 1, 1)
days = [first_day + timedelta(days=number) for number in range(366)]
names = ["temperature_daily_{:%Y_%m_%d}".format(day) for day in days]
expressions = [
    "{} = round(100 * (16.5 + 10.5 * cos(360 * ({} - 196) / 366.0)"
    " + 3 * sin(47 * {}) - 0.0065 * elev_state_500m"
    " - 0.00001 * (y() - 160000)))".format(name, number, number)
    for number, name in enumerate(names)
]
tools.r_mapcalc(expression="\n".join(expressions))

tgis.init()
dbif = tgis.SQLDatabaseInterfaceConnection()
dbif.connect()

temperature = tgis.open_new_stds(
    name="temperature_daily",
    type="strds",
    temporaltype="absolute",
    title="Daily mean air temperature",
    descr="Daily mean air temperature 2020 in 1/100 degree Celsius",
    semantic="mean",
    overwrite=True,
    dbif=dbif,
)

mapset = tgis.get_current_mapset()
for day, name in zip(days, names, strict=True):
    layer = tgis.RasterDataset("{}@{}".format(name, mapset))
    layer.load()
    layer.set_absolute_time(day, day + timedelta(days=1))
    if layer.is_in_db(dbif=dbif):
        layer.update(dbif=dbif)
    else:
        layer.insert(dbif=dbif)
    temperature.register_map(layer, dbif=dbif)

temperature.update_from_registered_maps(dbif=dbif)
start, end = temperature.get_absolute_time()
print("Dataset:", temperature.get_id())
print("Maps:", temperature.metadata.get_number_of_maps())
print("Time:", start, "to", end)
print("Granularity:", temperature.get_granularity())
print("Smallest minimum:", temperature.metadata.get_min_min())
print("Largest maximum:", temperature.metadata.get_max_max())
dbif.close()
```

Listing 4: Output of `register.py`

```
Dataset: temperature_daily@temporal_design
Maps: 366
Time: 2020-01-01 00:00:00 to 2021-01-01 00:00:00
Granularity: 1 day
```

```
Smallest minimum: -993.0
Largest maximum: 3144.0
```

5.2 Temporal aggregation

Listing 5 aggregates the daily maps to monthly means in degree Celsius. A granule is a `RasterDataset` without a map: it has only a time stamp, here one month. The topology builder relates the twelve granules to the daily maps, `r.series` averages the days that a granule contains, and `r.mapcalc` divides by 100. Each result map gets the time stamp of its granule and is registered in a new STRDS. The objects of `get_registered_maps_as_objects()` carry the identifier and the extents only, so the script reads the metadata with `select()` before it prints the range of each month.

Listing 5: Aggregation of daily to monthly temperature
`python/grass/docs/src/temporal_framework_design/examples/aggregate.py`
attached to this PDF as `aggregate.py`

```
"""Aggregate the daily temperature of 2020 to monthly means in degree Celsius

Needs the dataset temperature_daily of register.py in the current mapset.
"""

from datetime import datetime

import grass.temporal as tgis
from grass.tools import Tools

tools = Tools()
tools.g_region(raster="elev_state_500m", res=1000, flags="a")

tgis.init()
daily = tgis.open_old_stds("temperature_daily", "strds")
monthly = tgis.open_new_stds(
    name="temperature_monthly",
    type="strds",
    temporaltype="absolute",
    title="Monthly mean air temperature",
    descr="Monthly mean air temperature 2020 in degree Celsius",
    semantic="mean",
    overwrite=True,
)

dbif = tgis.SQLDatabaseInterfaceConnection()
dbif.connect()

# A granule is a map object without a map: it has only a time stamp.
granules = []
start = datetime(2020, 1, 1)
for _month in range(12):
    end = tgis.increment_datetime_by_string(start, "1 month")
    granule = tgis.RasterDataset(None)
    granule.set_absolute_time(start, end)
    granules.append(granule)
    start = end

daily_maps = daily.get_registered_maps_as_objects(dbif=dbif)
builder = tgis.SpatioTemporalTopologyBuilder()
builder.build(mapsA=granules, mapsB=daily_maps)

mapset = tgis.get_current_mapset()
for granule in granules:
    start, end = granule.get_absolute_time()
    name = "temperature_monthly_{:%Y_%m}".format(start)
    days = [layer.get_id() for layer in granule.get_contains()]
    print("{}: {} days".format(name, len(days)))
```

```
    tools.r_series(input=days, output="daily_average", method="average", overwrite=True)
    tools.r_mapcalc(
        expression="{} = daily_average / 100.0".format(name), overwrite=True
    )

    layer = tgis.RasterDataset("{}@{}".format(name, mapset))
    layer.load()
    layer.set_temporal_extent(granule.get_temporal_extent())
    if layer.is_in_db(dbif=dbif):
        layer.update(dbif=dbif)
    else:
        layer.insert(dbif=dbif)
    monthly.register_map(layer, dbif=dbif)

monthly.update_from_registered_maps(dbif=dbif)
for layer in monthly.get_registered_maps_as_objects(dbif=dbif):
    # The list holds the identifier and the extents; select() reads the rest.
    layer.select(dbif=dbif)
    start, end = layer.get_absolute_time()
    print(
        "{} {:%Y-%m-%d} {:%Y-%m-%d} {} {}".format(
            layer.get_name(),
            start,
            end,
            layer.metadata.get_min(),
            layer.metadata.get_max(),
        )
    )
dbif.close()
```

Listing 6: Output of `aggregate.py`

```
temperature_monthly_2020_01: 31 days
temperature_monthly_2020_02: 29 days
temperature_monthly_2020_03: 31 days
temperature_monthly_2020_04: 30 days
temperature_monthly_2020_05: 31 days
temperature_monthly_2020_06: 30 days
temperature_monthly_2020_07: 31 days
temperature_monthly_2020_08: 31 days
temperature_monthly_2020_09: 30 days
temperature_monthly_2020_10: 31 days
temperature_monthly_2020_11: 30 days
temperature_monthly_2020_12: 31 days
temperature_monthly_2020_01 2020-01-01 2020-02-01 -6.917096774193549 7.599677419354839
temperature_monthly_2020_02 2020-02-01 2020-03-01 -5.284137931034483 9.22896551724138
temperature_monthly_2020_03 2020-03-01 2020-04-01 -1.6048387096774193 12.911612903225807
temperature_monthly_2020_04 2020-04-01 2020-05-01 3.7060000000000004 18.222
temperature_monthly_2020_05 2020-05-01 2020-06-01 8.768064516129032 23.28548387096774
temperature_monthly_2020_06 2020-06-01 2020-07-01 12.608333333333333 27.125
temperature_monthly_2020_07 2020-07-01 2020-08-01 13.803225806451612 28.318387096774195
temperature_monthly_2020_08 2020-08-01 2020-09-01 12.27774193548387 26.793225806451616
temperature_monthly_2020_09 2020-09-01 2020-10-01 8.487333333333334 23.002333333333333
temperature_monthly_2020_10 2020-10-01 2020-11-01 3.160967741935484 17.676129032258064
temperature_monthly_2020_11 2020-11-01 2020-12-01 -1.8953333333333333 12.619333333333334
temperature_monthly_2020_12 2020-12-01 2021-01-01 -5.687741935483871 8.828064516129032
```

5.3 Aggregation with the algebra core

Listing 7 computes the same monthly means with the algebra core: `granules()` gives the months, `gather()` averages the days that each month contains with `r.series`, and the division by 100 is a slot by slot operation. `materialize()` computes the maps and registers them. The values are the same as those of listing 5.

Listing 7: Monthly means with the algebra core
python/grass/docs/src/temporal_framework_design/examples/algebra.py
attached to this PDF as algebra.py

```
"""Compute the monthly mean temperature of 2020 with the algebra core

Needs the dataset temperature_daily of register.py in the current mapset.
The result equals the one of aggregate.py.
"""

import grass.temporal as tgis
from grass.temporal import algebra as ta
from grass.tools import Tools

tools = Tools()
tools.g_region(raster="elev_state_500m", res=1000, flags="a")

tgis.init()
daily = ta.source("temperature_daily")
months = ta.granules("2020-01-01", "2021-01-01", "1 month")
averages = ta.gather(months, ta.contains, daily, reduce=ta.series_method("average"))
monthly = averages / 100.0

# Only materialize() computes maps: it runs r.series and r.mapcalc for each
# month and registers the results in a new space time raster dataset.
ta.materialize(
    monthly, "temperature_core", basename="temperature_core", time_suffix="gran"
)
core = tgis.open_old_stds("temperature_core", "strds")
dbif = tgis.SQLDatabaseInterfaceConnection()
dbif.connect()
for layer in core.get_registered_maps_as_objects(dbif=dbif):
    # The list holds the identifier and the extents; select() reads the rest.
    layer.select(dbif=dbif)
    start, end = layer.get_absolute_time()
    print(
        "{} {:%Y-%m-%d} {:%Y-%m-%d} {} {}".format(
            layer.get_name(),
            start,
            end,
            layer.metadata.get_min(),
            layer.metadata.get_max(),
        )
    )
dbif.close()
```

Listing 8: Output of algebra.py

```
temperature_core_2020_01 2020-01-01 2020-02-01 -6.917096774193549 7.599677419354839
temperature_core_2020_02 2020-02-01 2020-03-01 -5.284137931034483 9.22896551724138
temperature_core_2020_03 2020-03-01 2020-04-01 -1.6048387096774193 12.911612903225807
temperature_core_2020_04 2020-04-01 2020-05-01 3.7060000000000004 18.222
temperature_core_2020_05 2020-05-01 2020-06-01 8.768064516129032 23.28548387096774
temperature_core_2020_06 2020-06-01 2020-07-01 12.608333333333333 27.125
temperature_core_2020_07 2020-07-01 2020-08-01 13.803225806451612 28.318387096774195
temperature_core_2020_08 2020-08-01 2020-09-01 12.27774193548387 26.793225806451616
temperature_core_2020_09 2020-09-01 2020-10-01 8.487333333333334 23.002333333333333
temperature_core_2020_10 2020-10-01 2020-11-01 3.160967741935484 17.676129032258064
temperature_core_2020_11 2020-11-01 2020-12-01 -1.8953333333333333 12.619333333333334
temperature_core_2020_12 2020-12-01 2021-01-01 -5.687741935483871 8.828064516129032
```

5.4 Sampling a raster time series at vector points

Listing 9 samples the monthly means at three climate stations of the dataset: on the coast, in the Piedmont and in the mountains. It copies the stations into the mapset with v.extract, reads their coordinates with

pygrass and reads the raster cell under each station directly from each monthly map.

Listing 9: Sampling at climate stations
`python/grass/docs/src/temporal_framework_design/examples/sample.py`
attached to this PDF as `sample.py`

```
"""Sample the monthly mean temperature of 2020 at three climate stations

Needs the dataset temperature_monthly of aggregate.py in the current mapset.
The stations are copied from precip_30ynormals of the North Carolina sample
dataset; the raster cells are read directly with pygrass.
"""

import grass.temporal as tgis
from grass.pygrass.gis.region import Region
from grass.pygrass.raster import RasterRow
from grass.pygrass.utils import coor2pixel
from grass.pygrass.vector import VectorTopo
from grass.tools import Tools

tools = Tools()
tools.g_region(raster="elev_state_500m", res=1000, flags="a")
# On the coast, in the Piedmont and in the mountains
tools.v_extract(
    input="precip_30ynormals",
    output="stations",
    where="station IN (311606, 317924, 313565)",
)

stations = VectorTopo("stations")
stations.open("r")
points = []
for point in stations.viter("points"):
    station = int(point.attrs["station"])
    print(
        "Station {} at {:.0f} m: east {:.0f}, north {:.0f}".format(
            station, point.attrs["elev"], point.x, point.y
        )
    )
    points.append((station, point))
stations.close()

tgis.init()
monthly = tgis.open_old_stds("temperature_monthly", "strds")
region = Region()
for layer in monthly.get_registered_maps_as_objects(order="start_time"):
    start, end = layer.get_absolute_time()
    raster = RasterRow(layer.get_name())
    raster.open("r")
    for station, point in points:
        row, col = coor2pixel(point.coords(), region)
        value = raster[int(row)][int(col)]
        print("{:%Y-%m} {} {:.2f}".format(start, station, value))
    raster.close()
```

Listing 10: Output of `sample.py`

```
Station 311606 at 2 m: east 856070, north 140165
Station 313565 at 1615 m: east 354602, north 265290
Station 317924 at 186 m: east 567804, north 223132
2020-01 311606 6.32
2020-01 313565 -4.38
2020-01 317924 4.27
2020-02 311606 7.95
2020-02 313565 -2.75
2020-02 317924 5.90
```

```
2020-03 311606 11.63
2020-03 313565 0.93
2020-03 317924 9.58
2020-04 311606 16.94
2020-04 313565 6.24
2020-04 317924 14.89
2020-05 311606 22.00
2020-05 313565 11.31
2020-05 317924 19.95
2020-06 311606 25.84
2020-06 313565 15.15
2020-06 317924 23.79
2020-07 311606 27.04
2020-07 313565 16.34
2020-07 317924 24.98
2020-08 311606 25.51
2020-08 313565 14.82
2020-08 317924 23.46
2020-09 311606 21.72
2020-09 313565 11.03
2020-09 317924 19.67
2020-10 311606 16.39
2020-10 313565 5.70
2020-10 317924 14.34
2020-11 311606 11.34
2020-11 313565 0.64
2020-11 317924 9.29
2020-12 311606 7.54
2020-12 313565 -3.15
2020-12 317924 5.49
```

5.5 Spatio-temporal intersection of trajectories

The version of 2017 intersected the GPS tracks of two zebras. Here the tracks are generated from the dataset: two field crews sample the five stream gauges of Crabtree Creek in Raleigh (`usgsgages`) on one morning in opposite order, so that they meet on the way (listing 11). The positions are used directly, without vector maps: each becomes a `VectorDataset` without a map, whose time stamp is the time of the position and whose spatial extent is the position (listing 13). Snapping makes each position last until the next one and the last one for the granularity of the positions, five minutes; buffering widens a position by half the distance to the next position, at least by 50 m. The topology builder relates the positions of the two crews in two dimensions; a pair that is related in space and in time is a meeting, printed with the time and the area of the intersection.

Listing 11: The tracks of two field crews
`python/grass/docs/src/temporal_framework_design/examples/tracks.py`
attached to this PDF as `tracks.py`

```
"""Write the GPS tracks of two field crews that visit the Crabtree Creek gauges

The crews sample the five stream gauges of Crabtree Creek in Raleigh, taken
from usgsgages of the North Carolina sample dataset, on the same morning:
crew A from upstream to downstream, crew B the other way round, starting ten
minutes later. Each crew stays 30 minutes at a gauge and drives at 20 km/h
between two gauges. A position is recorded every 5 minutes.
"""

import csv
import math
from datetime import datetime, timedelta

from grass.tools import Tools

STEP = timedelta(minutes=5)
METERS_PER_STEP = 20000 / 12
```

```
STEPS_AT_GAUGE = 6

tools = Tools()
lines = tools.v_out_ascii(
    input="usgsgages",
    columns="SITE_ID",
    where="SITE_NAME LIKE 'Crabtree Creek%'",
    format="point",
).text.splitlines()
# USGS site numbers grow downstream.
rows = [line.split("|") for line in lines]
gauges = [
    (float(east), float(north))
    for east, north, _cat, _site in sorted(rows, key=lambda row: row[3])
]


def track(stops, start):
    """The positions (east, north, time) of a crew that visits the stops"""
    positions = []
    time = start
    for number, (east, north) in enumerate(stops):
        for _step in range(STEPS_AT_GAUGE):
            positions.append((east, north, time))
            time += STEP
        if number + 1 == len(stops):
            break
        next_east, next_north = stops[number + 1]
        distance = math.hypot(next_east - east, next_north - north)
        steps = max(1, round(distance / METERS_PER_STEP))
        for step in range(1, steps):
            fraction = step / steps
            positions.append(
                (
                    east + fraction * (next_east - east),
                    north + fraction * (next_north - north),
                    time,
                )
            )
            time += STEP
    return positions


crews = {
    "crew_a.csv": track(gauges, datetime(2020, 7, 15, 8, 0)),
    "crew_b.csv": track(gauges[::-1], datetime(2020, 7, 15, 8, 10)),
}
for filename, positions in crews.items():
    with open(filename, "w", newline="") as output:
        writer = csv.writer(output)
        for east, north, time in positions:
            writer.writerow(
                ["{:.1f}".format(east), "{:.1f}".format(north), time.isoformat(" ")]
            )
    print(
        "{}: {} positions from {} to {}".format(
            filename, len(positions), positions[0][2], positions[-1][2]
        )
    )
```

Listing 12: Output of `tracks.py`

```
crew_a.csv: 34 positions from 2020-07-15 08:00:00 to 2020-07-15 10:45:00
crew_b.csv: 34 positions from 2020-07-15 08:10:00 to 2020-07-15 10:55:00
```

Listing 13: Where and when the crews met
python/grass/docs/src/temporal_framework_design/examples/trajectories.py
attached to this PDF as `trajectories.py`

```
"""Find where and when the two field crews of tracks.py met

Each GPS position becomes a vector map object without a map: its time stamp
is the time of the position, its spatial extent the position. Snapping makes
each position last until the next one and the last one for the granularity
of the positions, five minutes; buffering widens a position by half the
distance to the next position, at least by 50 m. The topology builder then
relates the positions of the two crews in space and time.
"""

import csv
import math
from datetime import datetime

import grass.temporal as tgis

# The relations under which two positions share space and time
SPATIAL = ("EQUIVALENT", "COVER", "COVERED", "OVERLAP", "IN", "CONTAIN")
TEMPORAL = ("EQUAL", "DURING", "CONTAINS", "OVERLAPS", "OVERLAPPED")


def read_track(filename):
    """One VectorDataset object without a map for each position of a file"""
    positions = []
    with open(filename, newline="") as track:
        for east, north, time in csv.reader(track):
            position = tgis.VectorDataset(None)
            position.set_absolute_time(datetime.fromisoformat(time), None)
            position.set_spatial_extent_from_values(
                north=float(north),
                south=float(north),
                east=float(east),
                west=float(east),
                top=0,
                bottom=0,
            )
            positions.append(position)
    return positions


def buffer_positions(positions):
    """Widen each position by half the distance to the next one, at least 50 m"""
    for position, following in zip(positions, positions[1:] + [None], strict=True):
        distance = 0
        if following is not None:
            north, _south, east, *_rest = (
                position.spatial_extent.get_spatial_extent_as_tuple()
            )
            next_north, _south, next_east, *_rest = (
                following.spatial_extent.get_spatial_extent_as_tuple()
            )
            distance = math.hypot(next_east - east, next_north - north)
        position.spatial_buffer_2d(max(distance / 2, 50))


def temporal_relation(position, other):
    """The first relation of TEMPORAL under which other is related, or None"""
    relations = position.get_temporal_relations()
    for name in TEMPORAL:
        if other in relations.get(name, []):
            return name
    return None
```

```
tgis.init()
crew_a = read_track("crew_a.csv")
crew_b = read_track("crew_b.csv")
for positions in (crew_a, crew_b):
    tgis.AbstractSpaceTimeDataset.snap_map_list(positions)
    buffer_positions(positions)

builder = tgis.SpatioTemporalTopologyBuilder()
builder.build(mapsA=crew_a, mapsB=crew_b, spatial="2D")

for position in crew_a:
    for spatial, others in position.get_spatial_relations().items():
        if spatial not in SPATIAL:
            continue
        for other in others:
            temporal = temporal_relation(position, other)
            if temporal is None:
                continue
            space = position.spatial_intersection(other)
            time = position.temporal_intersection(other)
            print(
                "{:%H:%M}-{:%H:%M} {} m2 {} {}".format(
                    time.get_start_time(),
                    time.get_end_time(),
                    round(space.get_area()),
                    spatial.lower(),
                    temporal.lower(),
                )
            )
```

Listing 14: Output of `trajectories.py`

```
09:20-09:25 10000 m2 equivalent equal
09:25-09:30 10000 m2 equivalent equal
09:30-09:35 10000 m2 equivalent equal
09:35-09:40 10000 m2 in equal
```

5.6 Benchmark

Listing 15 measures typical operations: the registration of maps, the selection of maps by SQL as rows and as objects, and the temporal topology with the count of the relations. The number of maps is the first argument; the output shown is that of 50 maps, as the tests run it. The counts go to standard output, the times to standard error.

Listing 15: Benchmark
`python/grass/docs/src/temporal_framework_design/examples/benchmark.py`
attached to this PDF as `benchmark.py`

```
"""Measure typical operations of the temporal framework on many maps

Usage: python3 benchmark.py [number of maps]

The number of maps defaults to 50. The counts go to standard output, the
times to standard error, so that the output of two runs can be compared.
"""

import sys
import time

import grass.temporal as tgis
from grass.tools import Tools
```

```
def measure(label, function):
    """Run function and write its run time with label to standard error"""
    start = time.perf_counter()
    result = function()
    print(
        "{}: {:.3f} s".format(label, time.perf_counter() - start),
        file=sys.stderr,
    )
    return result


number_of_maps = int(sys.argv[1]) if len(sys.argv) > 1 else 50

# The benchmark measures the temporal framework, so the maps are small.
tools = Tools()
tools.g_region(raster="elev_state_500m", res=50000, flags="a")
names = ["benchmark_{}".format(number) for number in range(number_of_maps)]
tools.r_mapcalc(
    expression="\n".join(
        "{} = {}".format(name, number) for number, name in enumerate(names)
    )
)
with open("benchmark_maps.txt", "w") as file_of_names:
    file_of_names.write("\n".join(names) + "\n")

tgis.init()
dataset = tgis.open_new_stds(
    name="benchmark",
    type="strds",
    temporaltype="absolute",
    title="Benchmark",
    descr="Benchmark with {} maps".format(number_of_maps),
    semantic="mean",
    overwrite=True,
)


measure(
    "Registration",
    lambda: tgis.register_maps_in_space_time_dataset(
        type="raster",
        name="benchmark",
        file="benchmark_maps.txt",
        start="2000-01-01",
        increment="1 second",
        interval=True,
    ),
)
where = "start_time >= '2000-01-01 00:00:00'"
rows = measure("SQL selection", lambda: dataset.get_registered_maps(where=where))
print("Selected rows:", len(rows))
maps = measure(
    "Selection as objects",
    lambda: dataset.get_registered_maps_as_objects(where=where),
)
print("Selected map objects:", len(maps))
relations = measure(
    "Topology and counting", lambda: dataset.count_temporal_relations(maps)
)
print("Follows:", relations["follows"])
print("Precedes:", relations["precedes"])
```

Listing 16: Output of `benchmark.py 50`

```
Selected rows: 50
```

```
Selected map objects: 50
Follows: 49
Precedes: 49
```

Table 1 gives the times of `benchmark.py 1000`, measured on 6 November 2026 on an Intel Core Ultra 9 285T with 24 cores, GRASS 8.6.0dev, Python 3.12 and SQLite 3.45: the median of three runs, each in a new mapset of the dataset. They show the order of magnitude on one computer, not a property of the framework.

Table 1: Times of `benchmark.py 1000`, median of three runs

Operation	Time [s]
Registration	19.4
SQL selection	0.011
Selection as objects	0.026
Topology and counting	0.026

References

[1] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11): 832–843, 1983. doi: 10.1145/182.358434.

[2] Sören Gebbert and Edzer Pebesma. A temporal GIS for field based environmental modeling. *Environmental Modelling & Software*, 53:1–12, 2014. doi: 10.1016/j.envsoft.2013.11.001.

[3] Sören Gebbert and Edzer Pebesma. The GRASS GIS temporal framework. *International Journal of Geographical Information Science*, 31(7):1273–1292, 2017. doi: 10.1080/13658816.2017.1306862.

[4] Markus Neteler and Helena Mitasova. *Open Source GIS: A GRASS GIS Approach*. Springer, New York, third edition, 2008.